

Memóriamenedzselés

Az előadás anyaga

Dr. Vadász Dénes: Operációs rendszerek és
URL linkek alapján készült el

Témakörök

- Memória, címtartomány fogalmak
- A címleképzés
- Memóriamenedzselési osztályok
- Valós MM
- Foglalási stratégiák
- Mintapéldák

OS osztályozási szempontok - ismétlés

„Memória menedzselés szerinti osztályozás:

Alapvetően megkülönböztetünk *valós címzésű* és *virtuális címzésű* rendszereket:

- *valós címzésű rendszereken* belül a *fix*, vagy *változó partíciókra* osztott memóriakezelés.

OS osztályozási szempontok - ismétlés

- *virtuális címzésű rendszerek* esetén:
 - ki/be söprő (swapping in/out) rendszerek,
 - ki/be lapozó (demand paging) rendszerek,
 - ki/be söprő és lapozó (swapping and paging) rendszerek.

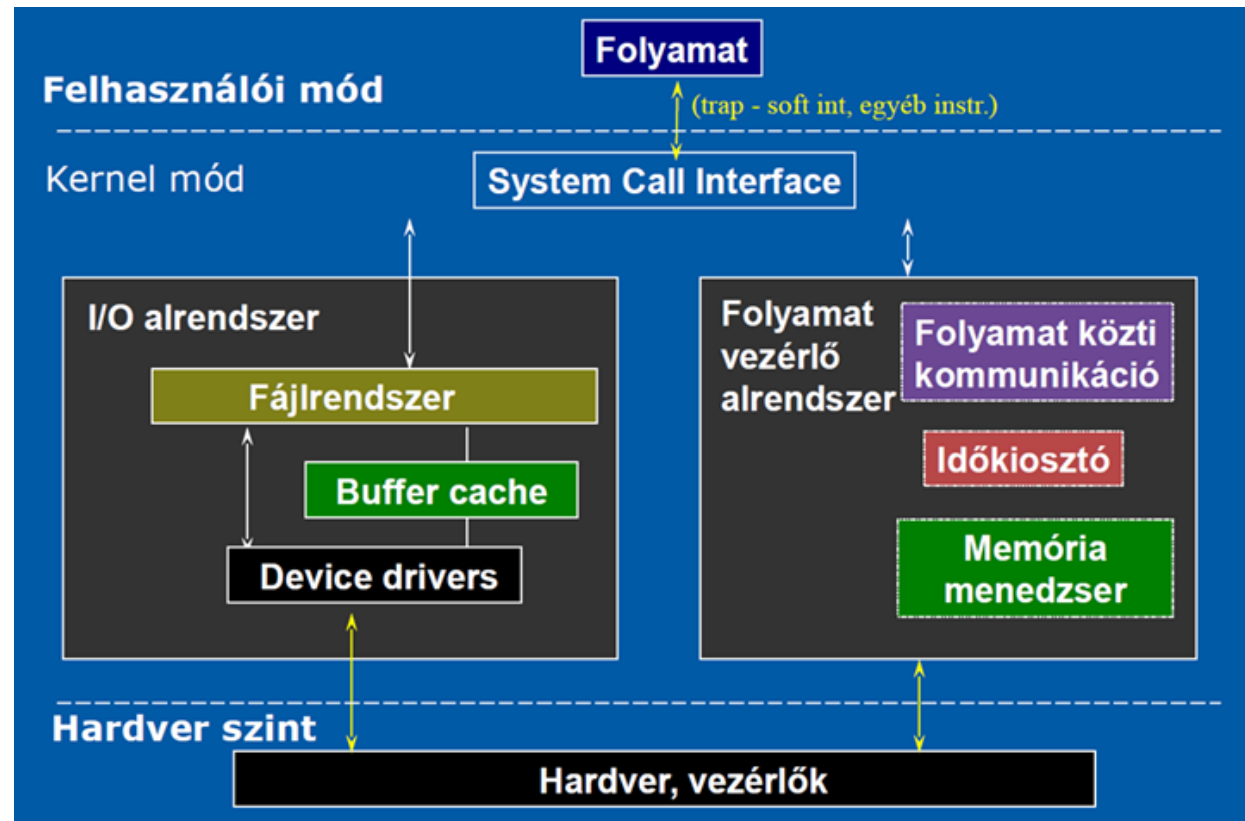
OS osztályozási szempontok - példák

- **MS DOS:** valós címzésű, változó partíciókra osztó rendszer.
- **Windows NT:** virtuális címzésű, lapozó rendszer.
- **UNIX:** virtuális címzésű rendszer.

A korai Unix-ok ki/be söprő rendszerek, ma már ki/be söprő és lapozó a memóriakezelés.

UNIX funkcionális felépítése - ismételés

Folyamatvezérlő alrendszer



Memória kezelés - ismételés

Számítógép architektúrák kurzus már tanulták:

- a memória *címezhető rekeszekből* (cellákból) áll,
- az *instrukciók is memória cellákban* vannak, (a PC/IP is *memória címre hivatkozik*)
- az *instrukciók* adatrészükben hivatkozhatnak memória címekre,
- az instrukciók *processzekhez* tartoznak.

Tárolóeszközök – memória

- Erőforrás, amit
 - *Igényelni* (kérni, allokálni) kell (a processzek igénylik),
 - *Védeni kell* (ellenőrzött elérések legyenek).
- A memória címekkel rendelkező cellák készlete (rekeszek: bájtok, szavak, rekordok).
- Egy-egy cella elérése (load-store) a címére való hivatkozással történhet.

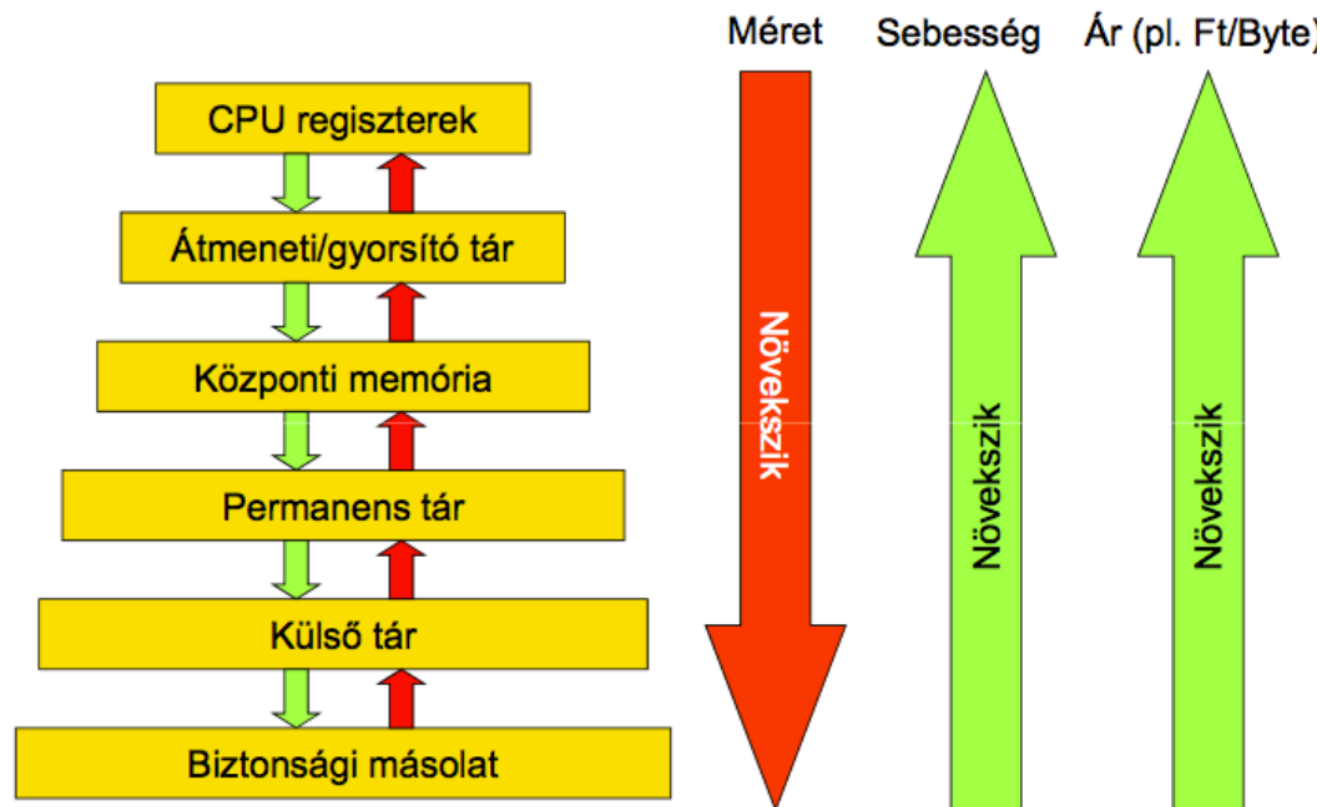
Tárolóeszközök – memória

A tárolóeszközök *hierarchikus szintekbe* szervezett:

- Processzorbeli-processzor közeli *gyorsítótár*.
- *Központi tár*.
- *Másodlagos tár*.

Tárolóeszközök hierarchikus felépítése

A táruk *hierarchikus szintekbe* szervezve.



Forrás: OPRE, BME-VIK

Regiszterek - ismételés

A regiszterek a processzorban található kis-méretű tárolók.

Általános célú regiszterek

- AX – akkumulátor: aritmetikai műveletek,
- BX – bázis regiszter: tömbindexelés,
- CX – számláló (count) regiszter: ciklusszervezés, ismétlések számolása,
- DX – adat (data) regiszter: I/O műveletek, aritmetikai műveletek.

Szegmens regiszterek

- CS – kód szegmens: futtatandó utasítások tárolása (maga a program),
- DS – adat szegmens: globális változók, konstansok tárolása,
- SS – verem szegmens: lokális változók tárolása, visszatérési cím,
- ES – extra szegmens: tetszőlegesen felhasználható terület (pl. heap).

Regiszterek - ismételés

Pointer (index) regiszterek

- IP – utasításszámláló, az éppen következő assembly utasításra mutat,
- SI – forrás (source) index, sztringműveletek kiinduló címe,
- DI – cél (destination) index, sztringműveletek érkezési címe,
- SP – verem mutató (stack pointer), a verem aktuális tetejét mutatja,
- BP – bázis pointer, bázis-index címzéshez (tömbindexelés).

https://www.inf.u-szeged.hu/~vargalg/oktatas/assembly_18t/tananyag/regiszterek.html

Regiszterek - ismétlés

Speciális regiszter (SR = FLAGS)

Státusz regiszter, minden bitje egy nevesített jelzőérték (true/false).

O - előjeles átvitel (overflow)

D - sztringműveletek iránya

I - interruptok engedélyezése

T - csapda (debug) eng.

S - eredménye előjele (sign)

C - eredménye átvitele (carry)

Z - eredménye nulla-e (zero)

A - félszavas átvitel (BCD)

P - eredmény páros/páratlan

A táruk hierarchikus felépítése - ismételés

- *Átmeneti/gyorsító tár (Cache):* jellemzők
- *Központi (fizikai) memória:* jellemzők
- *Permanens/háttértár (HDD, Flash):* jellemzők

Központi memória kezelése:

- *fizikai, logikai, virtuális memória* fogalma,
- ezeknek a megfeleltetése és kezelése.

Permanens tár kezelése: a virtuális tárkezelésen keresztül kapcsolódik majd a központi memória kezeléséhez.

Memória - mint erőforrás

A processz szemszögéből a *memória* egy erőforrás, amire szüksége van.

Egy véges mennyiségű erőforrás, ahol egy cellát egyszerre egy időben csak egy processz használhat (kivéve osztott mem.).

Akár el is vehető a processztől (hiszen lementhető az állapota) és gond nélkül vissza is adható.

Memória menedzselése - MM

*Erőforrás kezelés egy külön OS komponens segítségével:
Memory Management (MM).*

Az MM feladatai az erőforrás kezeléssel kapcsolatban:

- 1. Memória foglalások kezelése.*
- 2. Memória területek védelme.*
- 3. Osztott memória használat biztosítása.*

Memória, mint erőforrás

1. Memória foglалások kezelése

a) *lefoglalni* (allokálni) a processz számára szükséges memória területeket.

- *statikus allokálás*: A processz születésekor a kontextusa számára – ismétlés.
- *dinamikus allokálás*: A processz élete során felmerült újabb igények teljesítése – ismétlés.

Processz kontextus - ismétlés

Mi a processz *kontextus tartalma*?

- a program *kódszegmensei* (instrukciók sorozatai),
- a program *adatszekciói* (inicializált adatszekciók - kezdőérték beállítás), konstans szekciók stb.),
- a processz *veremtárai* (stack), az argumentumátadáshoz,
- *regiszter állapotok*,
- a processz *menedzselési információi*: *azonosítási, erőforrási és állapotokat leíró, hozzáférési információk*.

Memória, mint erőforrás

b) *Nyilvántartani* a processz számára *lefoglalt* területeket, valamint a *szabad területeket*.

c) Biztosítani lehetőségét az *extrém használatra* is.

Pl.: olyan processzek is futhassanak, amely *több memóriát igényelnek*, mint az *összes fizikai memória*.

Memória, mint erőforrás

2. Memória területek védelme

Egyik processz se férhessen hozzá egy másik processzhez rendelt memória területhez.

3. Osztott memória használat biztosítása

A processzek használhassanak igény esetén *közös memória területeket*.

Lehetséges legyen *kódmegosztás, adatmegosztás* – pl. *szálak, task* kezelése.

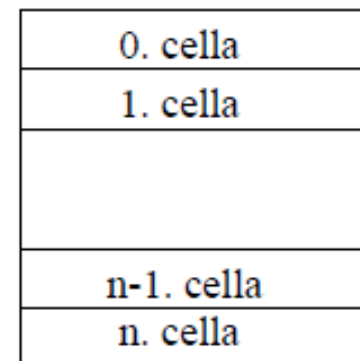
Címtartományok - ismétlés

- A memória - címekkel rendelkező rekeszek készlete, fontos fogalom a *címtartomány* fogalom.
- A címtartomány *a címek készlete* (szemben a memóriával, ami rekeszek készlete).

• A címtartomány modellje



• A memória modellje



Címtartományok - ismétlés

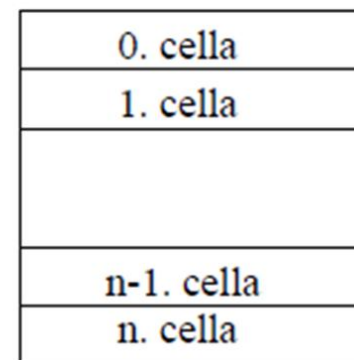
A címtartományok lehetnek:

- Logikai címtartományok
- Fizikai címtartományok

• **A címtartomány modellje**



• **A memória modellje**



Logikai címtartományok

A processzekhez *logikai címtartományok* tartoznak (Logical Address Space)

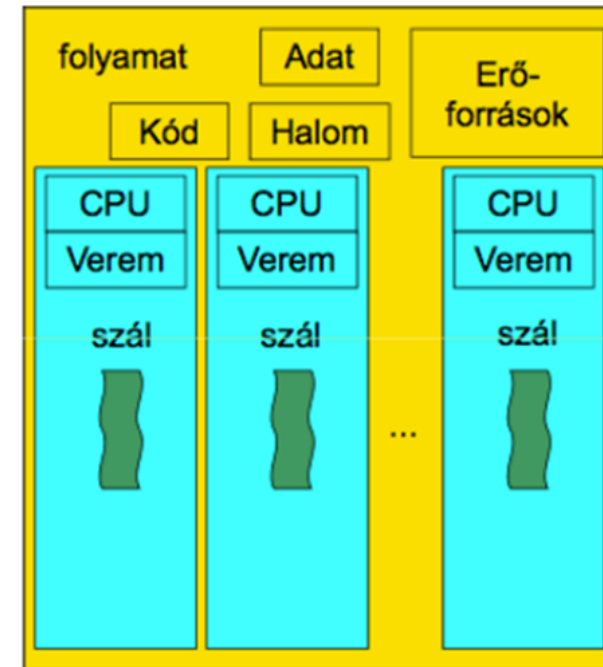
- Az ő szemszögükből nézve van *saját processzoruk*,
- Van saját (*logikai*) *memóriájuk*.
- A processzekhez egy *folytonos címtartományt* szoktak képzelni.

Logikai címtartományok

Már a *logikai címtartományokat* is érdemes *szakaszolni*:

- címek a *kódnak*,
- az *adatoknak*,
- a *vermeknek*.

Logikai cím: A CPU generálja a processz futása közben.



Fizikai címtartományok

A CPU a *fizikai memóriából* tud betölteni, ill. letárolni adatokat.

- A fizikai memóriához *fizikai címtartományok* tartoznak.
- A *címsín* szélességétől függ, mekkora a *fizikai címtartomány*.

Logikai címek

A logikai címek lehetnek:

- *relatív címek, vagy*
- *virtuális címek.*

A logikai címek és a fizikai címek közötti megfeleltetést (átalakítás) a címleképzésnek (mapping) nevezzük.

Logikai címek - címleképzés

A mai rendszerekben a processzek kontextusában *logikai címek* vannak (többnyire virtuális címek).

- *A logikai címből - fizikai címre való leképezését, közben a cím köté*~~ődés~~*s megoldását a MM alrendszer a hardver.*

MMU egységével együtt oldja meg:

- *A címleképzés – köté*~~ődés~~*s során gondoskodnak a védelemről.*
- Szükség esetén a *ki-be mozgatásról* (tárcseréről).

Memóriamenedzsment osztályok

Memória menedzselő rendszerek - több szempont szerint is osztályozhatunk.

Egyik osztályozás szerint a memória menedzselő rendszerek:

- mozgatják a *processzek kontextusát*
 - ki-belapozó rendszerek,
 - ki-besöpítő rendszerek.
- nem mozgatják a kontextusokat.

Memóriamenedzsment osztályok

Másik osztályozási szempont - **Memória címzésű rendszerek szerint:**

1. *valós címzésű rendszerek:*

- monoprogramozású,
- multiprogramozású rendszerek,
 - fix partíció,
 - változó partíció rendszerek.

Memóriamendzsent osztályok

2. *virtuális címzésű rendszerek:*

- lapozós (paging) rendszerek,
- szegmens-leképző, ki-besöprő rendszerek,
- szegmentáló, ki-besöprő és ugyanakkor lapozó rendszerek.

Memória - Relatív címzésű rendszerek

A program készítésekor *relatív címek* keletkeznek.

A *relatív címek* egy *bázis címhez* képesti címek.

Eredmény:

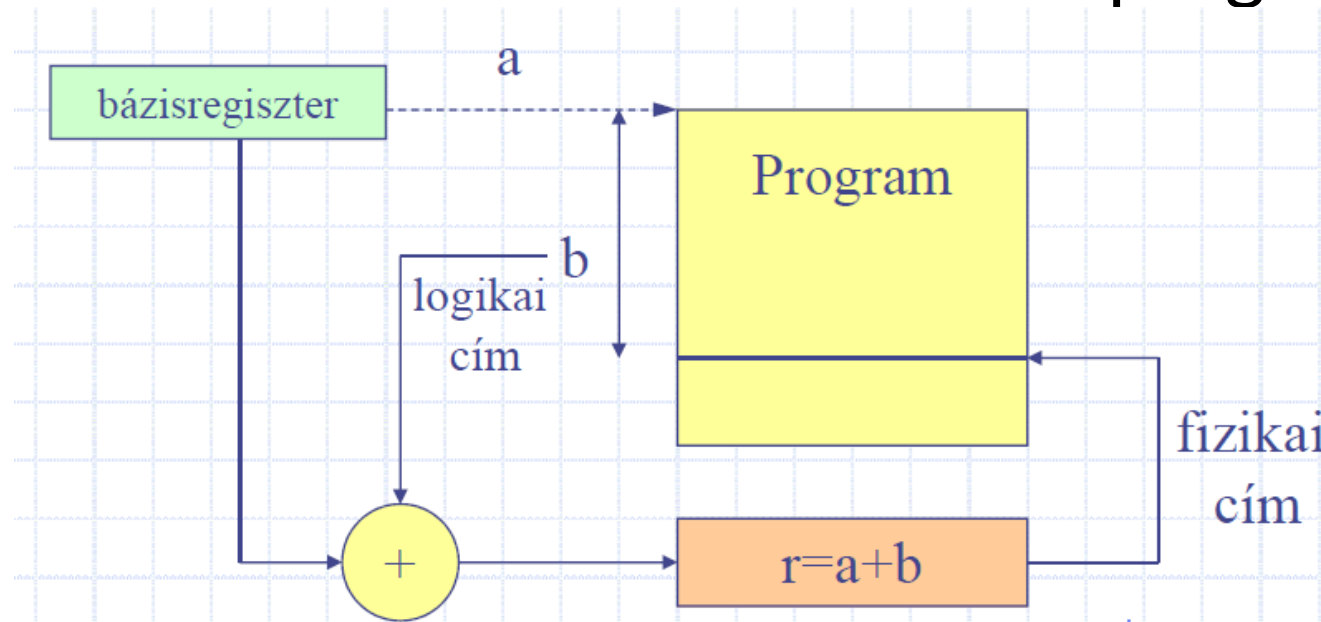
A processzek az egyik regiszterben tárolják a *bázis címet*.

Memória - Relatív címzésű rendszerek

Bázis-relatív címzés

A program tetszőleges helyre betölthető.

A bázisregisztert a *betöltési kezdőcímre állítva* a program végrehajtható.



Memória – Címköötődés - Relatív címzésű rendszerek

Előnyei:

- A processzek *áthelyezhetők*: tetszőleges *szabad területre betölthető* vagy szükség esetén más *területre áthelyezhető*.

Hátrányai:

- A processz csak *egybefüggő területen* helyezkedhet el!
- *Megkötés*: a processz *kevesebb memóriát használhat*, mint a *fizikai memória tartomány*.

Memória – Címköötődés -Virtuális címzésű rendszerek

Virtuális címzésű rendszerek

- A programok *saját virtuális címtartománnyal* rendelkeznek, *virtuális címeket* használnak.
- Az OS minden *virtuális címhez* biztosít egy *rekeszt (tárolót)*, ami nem *feltétlenül a memóriában* van, hanem bármilyen más *tároló eszközön* is lehet.

Memória - Címköötődés -Virtuális címzésű rendszerek

Címleképzés:

- *A virtuális címzésű rendszereknel minden címhivatkozás leképzésre szorul.*
- *A címleképzést támogatja a CPU MMU egysége.*

Memória - Címköötődés -Virtuális címzésű rendszerek

Címleképzés problémái:

- a leképzés *sokkal bonyolultabb*, mint a *relatív címzésű rendszerek* (lásd lapozásos, szegmentálásos)
- a címek csak *memória címekre képezhetők* le.

Előnye:

A programok *tetszőleges méretű virtuális címtartományt* használhatnak, akár a *fizikai memória címtartománytól* nagyobb is.

Memória -Címkötődés -Virtuális címezésű rendszerek

Hátrányai:

- Bonyolultabb a címképzés. (Lassul egy utasítás végrehajtásának a sebessége)
- A címképzésnél meg kell oldani, hogy a más tárolóra elhelyezett memóriaterületek betölthessenek.

Memória kezelés - MM fajták

Memória kezelés - MM fajták

Memória kezelés - MM fajták

MM fajták

- Single task-os modell – meg említjük
- **Fix partíció**
- **Változó partíció**
- Virtuális memória kezelés

Memória kezelés - Single task-os modell

Single task-os modell

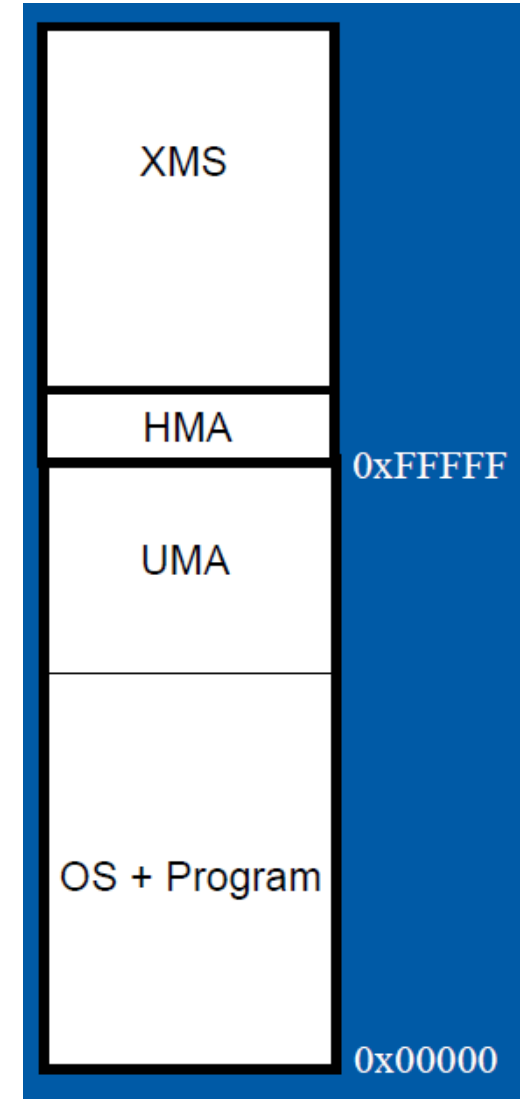
- Csak az *OS* és egy *taszk*.
- A korai rendszerekben *valós címzésű*, az *OS* számára mindig *fix méretű a terület*.
- Később *relatív címzésű*, *változó OS terület*.

AZ MS-DOS hasonló rendszert használt a 640kb-s korlát miatt.

Memória kezelés - Single task-os modell

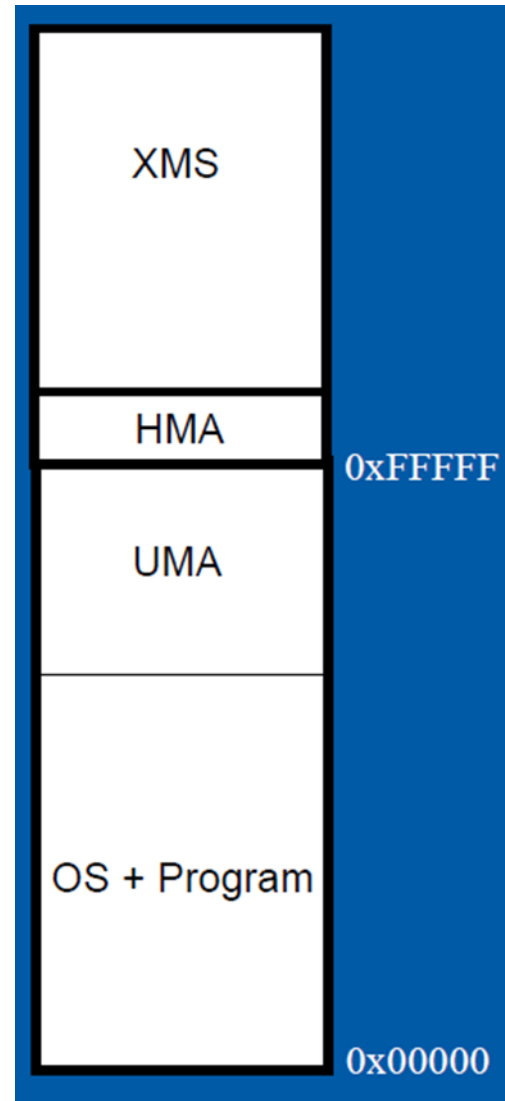
Single task-os modell

- pl. IBM PC + DOS
- Conventional memory (hagyományos)
 - 0 – 640Kbyte
- UMA (Upper Memory Area)
 - 640Kbyte – 1Mbyte
 - BIOS, VRAM, perifériák



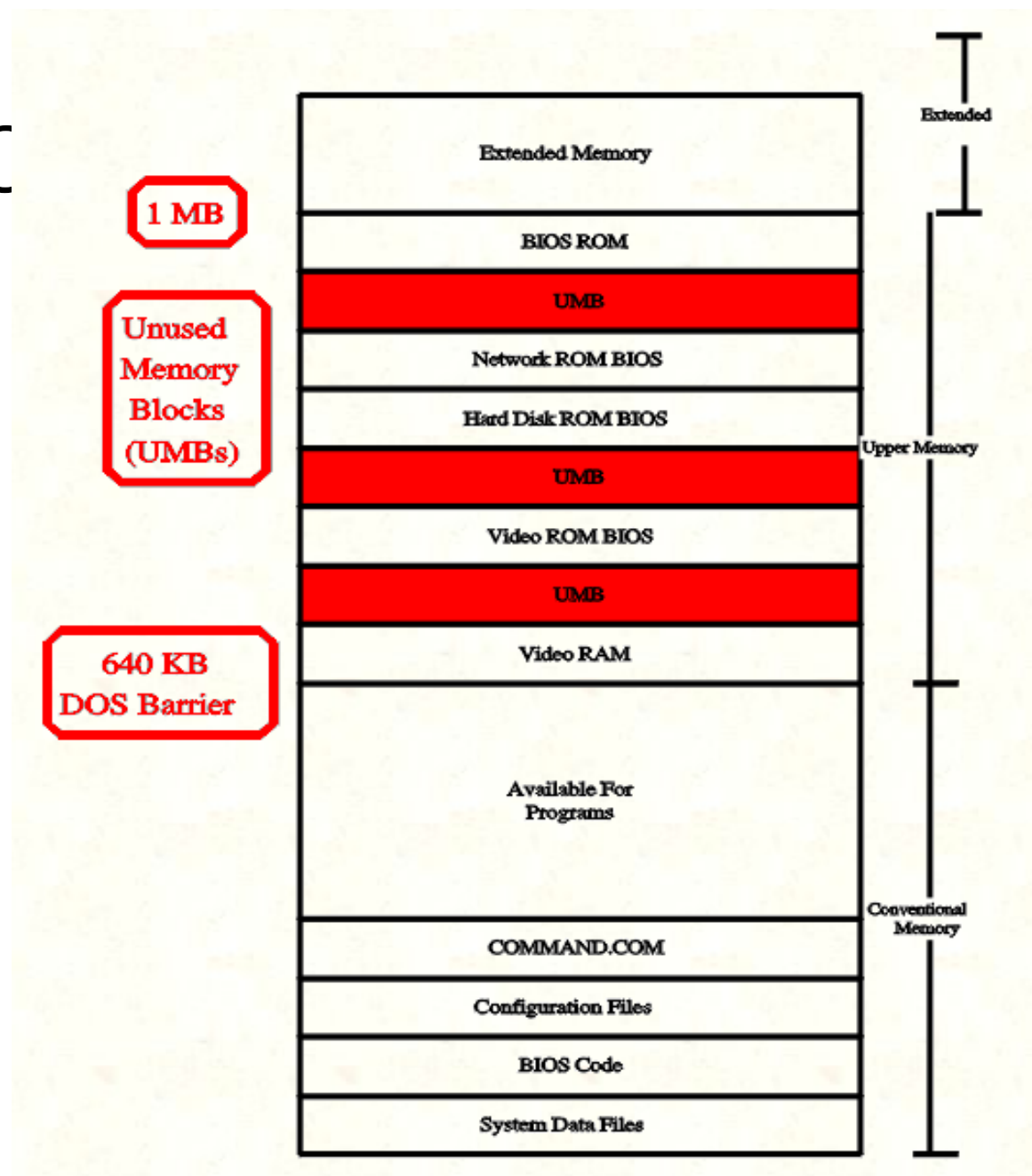
Memória kezelés - Single task-os modell

- XMS (eXtended Memory System)
1MB feletti rész, másolat
- HMA (High Memory Area)
1MB feletti kicsi rész a seg+off
címezésből fakadóan

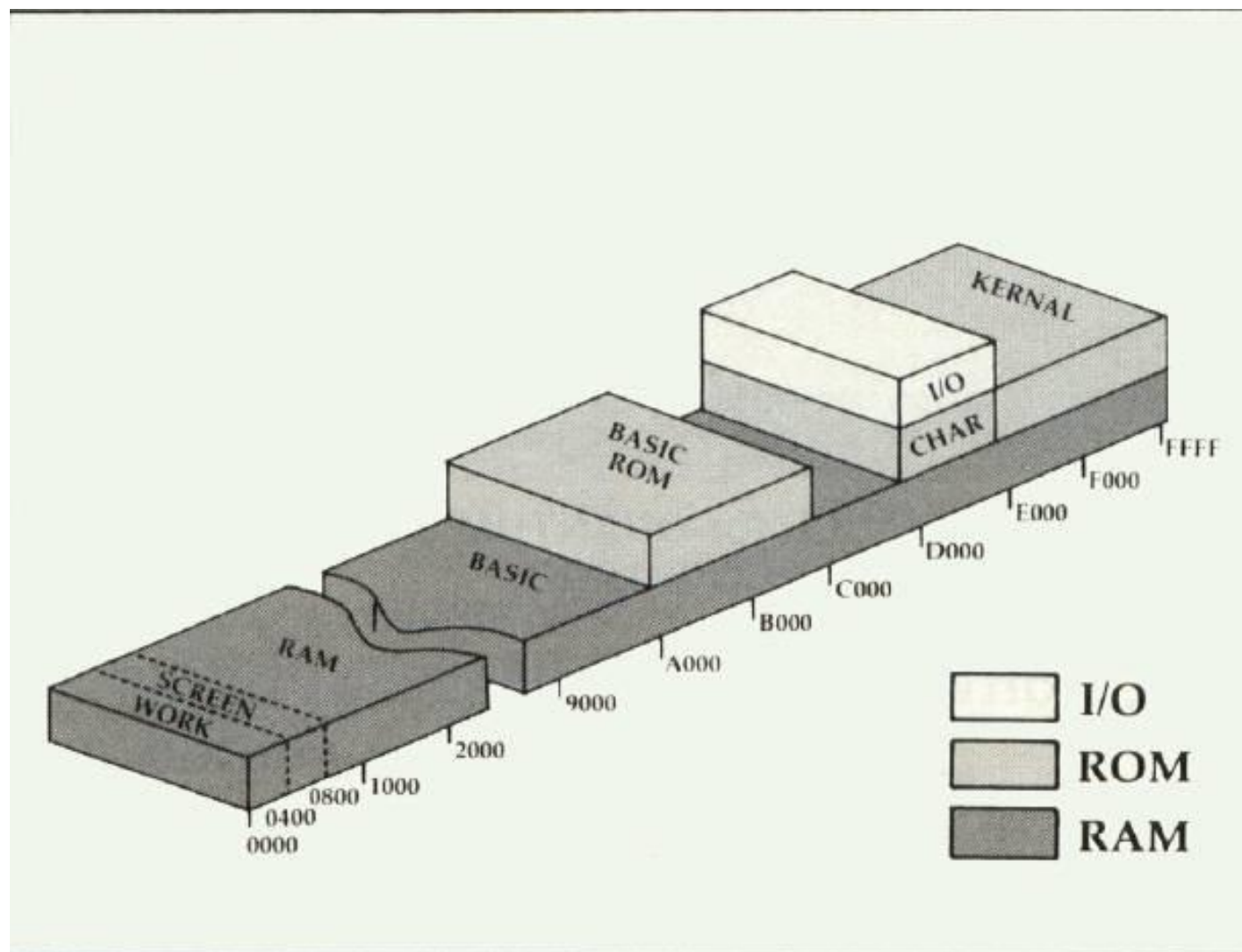


Memória kezelés – DC

DOS memória térkép



Memória kezelés – C64



Memória kezelés - Single task-os modell

Allokálás

Az OS feletti összes terület a processz rendelkezésére áll.

A dinamikus allokálás megvalósítható.

A fizikainál nagyobb memória igényű processz kezelésére az OS nincs felkészülve, de a programozó megoldhatja az un. *overlay* technikával.

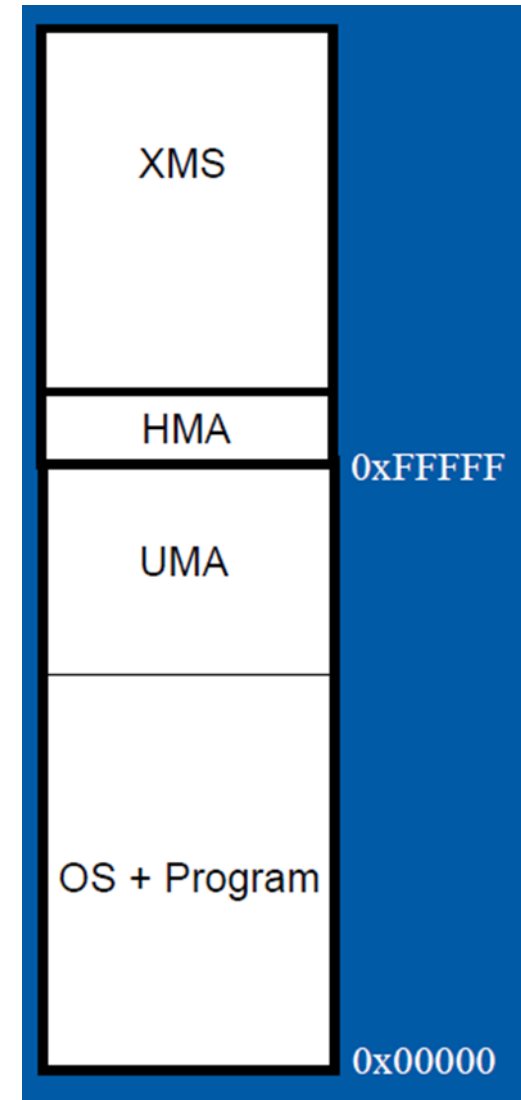
Memória kezelés - Single task-os modell

Védelem

Csak az OS területet *kell védeni a processztől.*

Határcím figyeléssel megoldható.

Osztott memória: nincs rá igény.



Memória kezelés - Overlay technika

Átfedő programrészek (overlay) technikája.

Olyan terjedelmes programok esetén használják, amelyek egyszerre nem férnének a számítógép fizikai memóriájába.

A programok egy *része* „szétvágható” olyan részekre, amelyek *időben elkülönülten* dolgoznak.

Ezeket szükség esetén *egyesével töltjük be*, majd használatuk után újra *a háttértárba menthetjük*, hogy a felszabaduló területre egy *másik overlay részt* tölthessünk.

Memória kezelés - Overlay technika

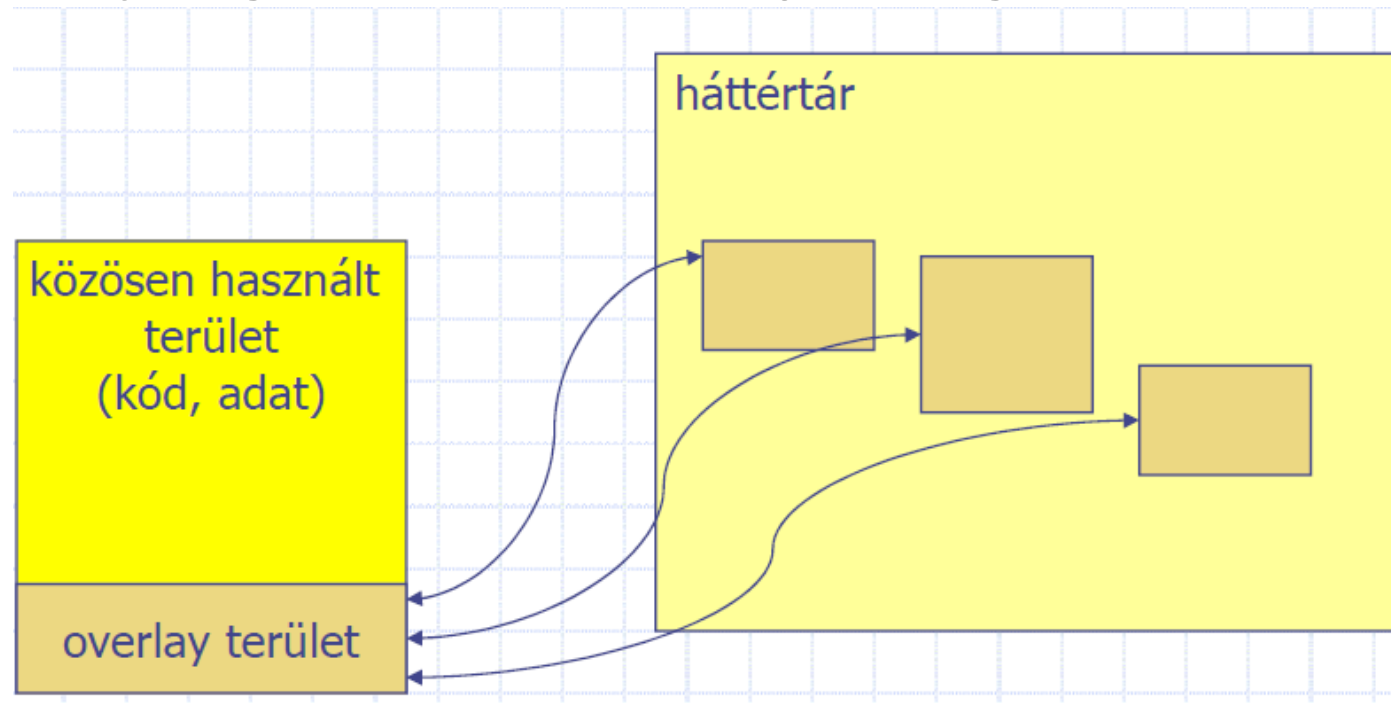
Overlay területre mindig a program csak éppen szükséges része töltődik be.

A program részekre bontása:

- közös adat- és programrészek (nem változik)
- olyan átfedő részek, amelyek közül egyidőben csak egyre van szükség.
- Az átfedés számára fenntartott tárterület a legnagyobb átfedő programrész hosszával egyenlő.
- Nincs szükség OS támogatásra.

Memória kezelés - Overlay technika

A módszer nem igényel OS támogatást (csak linker), *teljes egészében a programozó valósítja meg.*



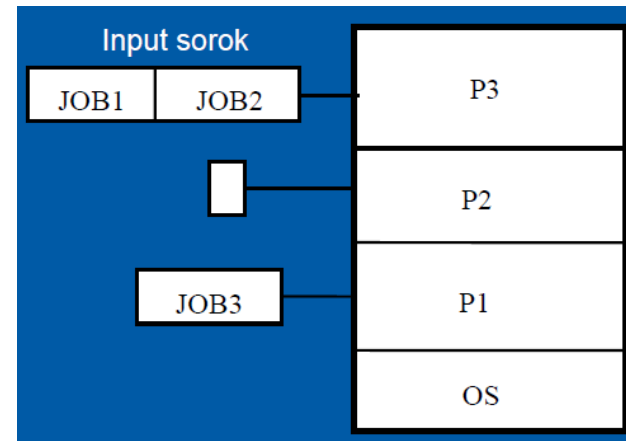
Memória kezelés – Fix partíciók

Fix partíciók

Az OS-en kívüli terület *fix méretű és darabszámú partíciókra* van felosztva (bootoláskor meghatározott).

(Pl. $2 \times 1\text{GB} + 4 \times 0.5\text{GB} + 4 \times 0.25\text{GB} + 8 \times 128\text{MB}$)

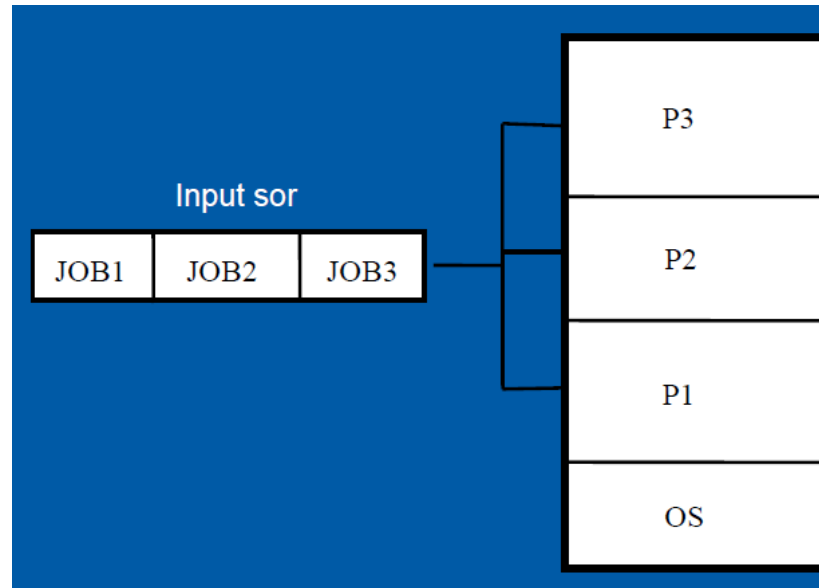
Egy processz = egy partíció!



Memória kezelés – Fix partíció

Fix partíció

Relatív címzést használ, ilyenkor közös várakozó sor, bár egy processz nem biztos, hogy bármelyik partícióban képes futni (de nem csak egyben).



Memória kezelés – Fix partíciók - Allokálás

Allokálás

Egy processz egy teljes partíciót megkap.

(meg akkor is ha az igénye csak töredéke volt).

A dinamikus allokáció addig megvalósítható, amíg a processz nem növi ki a partíciót.

Nagy méretű (a legnagyobb partíció méretétől nagyobb) processzek nem futtathatók, a programozó használhat *overlay technikát*.

Memória kezelés – Fix partíciós

Védelem

A védelem megoldása a *partíció határcímeinek figyelésével*.

(határcímelek egy-egy regiszterbe – bázisregiszter,
limitregiszter –, minden címhivatkozásnál ellenőrzés)

SHM

Osztott memóriát *csak OS területen* lehet biztosítani.

A hivatkozás *rendszerhívásokon* keresztül történik (nem lehet a szokásos *memória hivatkozás*)

Memória kezelés – Fix partíció

Előnyök:

- *Multitask-os* környezetben is használható.
- Nagyon *kis költségigény* (allokációs idő, nyilvántartás mérete, *stb.*).

Hátrányok:

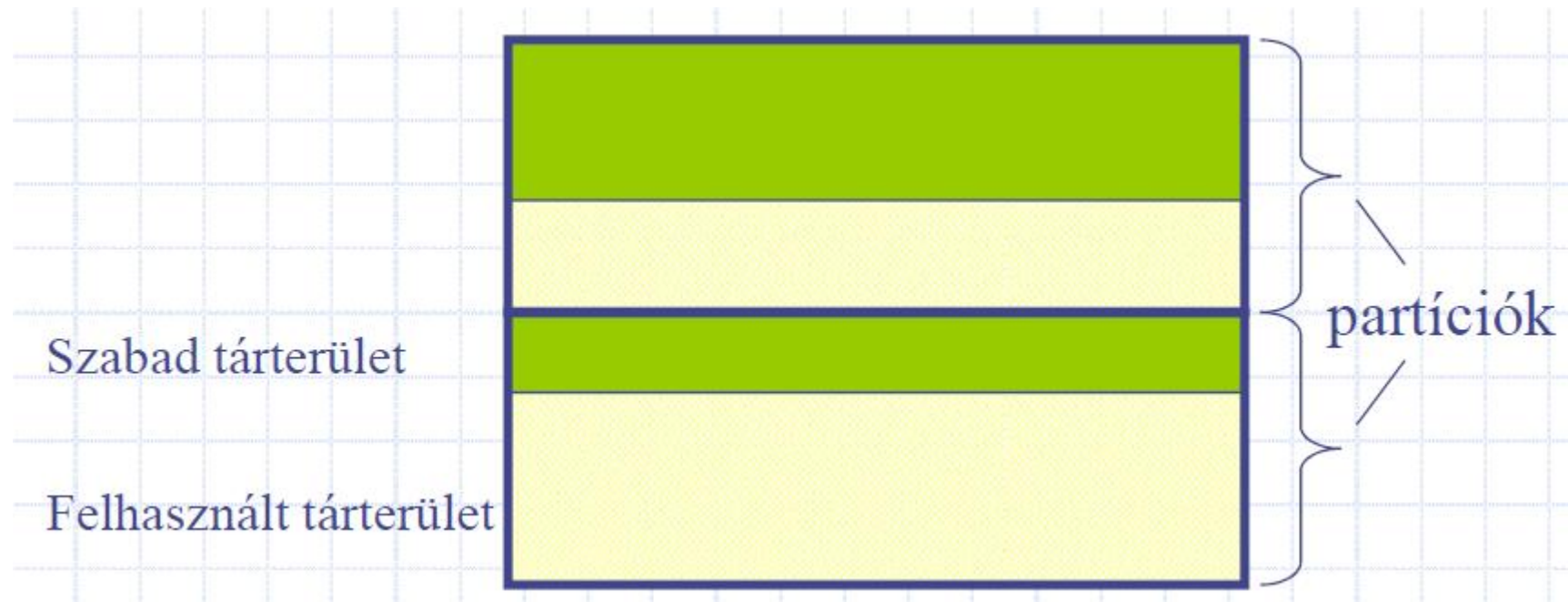
- Rossz helykihasználtság, *belső töredezettség* van!
- Korlátozott processz szám, méret.

Belső töredezettség: amikor egy memória partíción belül kihasználatlan területek vannak.

Memória kezelés – Fix partíciók

Belső töredezettség

A processzek nem használják ki a rendelkezésére bocsátott partíciót.

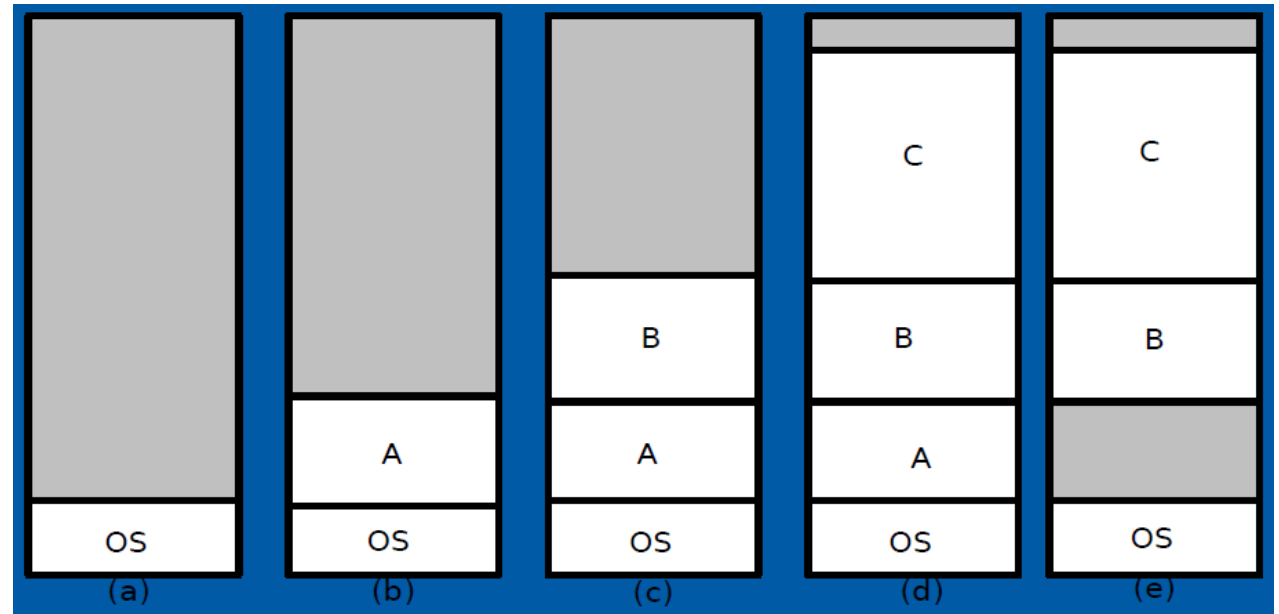


Memória kezelés - Változó partíciók

Változó partíciók

Továbbra is *egy processz egy partíció*, de nincsenek *rögzített számú és méretű partíciók*, a partíciók a *processzek igényeinek megfelelően* jönnek létre.

Relatív címzésű módszer.



Memória kezelés – foglalási stratégiák

Allokálás: Elhelyezés (foglalás) a szabad partíciók valamelyikébe.

*A szabad partícióból két partíció lesz az lefoglalt rész és egy új **szabad partíció**.*

Foglalási stratégiák:

- *Első megfelelő (First fit)*

A szabad területek közül a legelső elegendő méretűt foglaljuk le (gyors, kb. 30% a kihasználtság).

Memória kezelés – foglalási stratégiák

- *Következő megfelelő (Next Fit)*
 - A keresés/foglalást az *utoljára lefoglalt tartomány* végéről indul, az első megfelelő méretűt lefoglaljuk.
 - Igen gyors algoritmus, a memória 30%-a marad kihasználatlan.

Memória kezelés – foglalási stratégiák

- *Legjobban megfelelő (Best fit)*

A legkisebbet foglaljuk le azon szabad területek közül, amelyek legalább akkorák, mint a lefoglalandó terület. Minimális külső tördelődés.

A legkisebb még elegendő méretű.

Memória kezelés – foglalási stratégiák

Legrosszabban illeszkedő (Worst fit)

Az elérhető *legnagyobb szabad területet* allokáljuk.

A módszer az, hogy a maradék terület még talán elegendő lesz *egy újabb foglalás számára*.

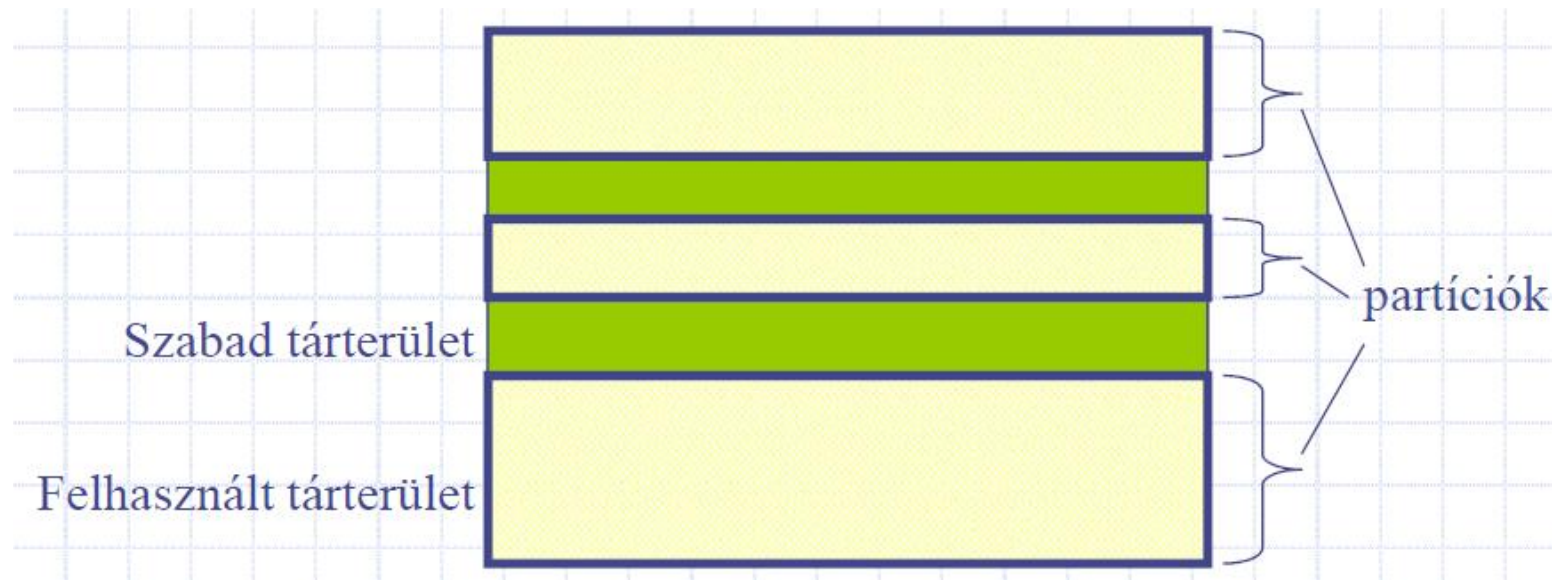
Tehát, a maradék helyre még befér majd valami.

Hatékonyság szempontjából a legrosszabban illeszkedő a leggyengébb, a többi nagyjából azonos.

Memória kezelés – Változó partíciók

Külső töredezettség

A szabad memória kis, egymással nem szomszédos részekre oszlik.



Memória kezelés – Változó partíció

Allokálás

A dinamikus allokáció problémát okoz. Ha a következő partíció szabad, akkor megnövelhető a partíció.

A processz befejeződése szintén problémát okoz: össze kell vonni az egymást követő üres partíciókat (lyukakat), különben a külső töredezettség nagyon nagy lesz.

A külső töredezettség megszüntetésére időnként teljes átrendezésre is szükség lehet.

Memória kezelés – Változó partíció

Partíció nyilvántartásra van szükség a szabad területek kereséséhez, a partíció határok nyilvántartásához.

A nyilvántartás lehet:

- *bittérképes*: a memória blokkokra van osztva minden blokkról 1 bit infó, hogy szabad-e.
- *láncolt listás*: minden *partíció* egy láncolt listán bejegyezve (kezdete, hossza, szabad-e)

Memória kezelés – Változó partíció

Védelem

A védelem itt is a *határcímek figyelésével oldható* meg. Két regiszter kell, amelyek a címtartomány határait tartalmazzák.

A határcímek a processz kontextusba (valamelyik regiszterbe – bázisregiszter, limitregiszter) letárolódnak.

Osztott memória

Az osztott memória itt is csak OS területen képezhető, *rendszerhívásokkal* kezelhető.

Memória kezelés – Változó partíció

Előnyök:

- Nincs a belső töredezettség.
- Jobb a helykihasználtság, mint a fix partíció (kb. 50%-os, függ a processzek átlagos memória igényétől)

Hátrányok:

- A processzek továbbra is csak egyben helyezhetők el.
- Sokkal több az *OS feladat*, mint a fix partíciósnál.

Memória kezelés – mintapélda

Foglalási stratégiák - mintapéldák

Adott egy rendszer, melyben a következő

Szabad területek: 24k, 29k, 9k, 19k, 69k, 39k és 39k.

Foglalási igény: 39k, 35k, 28k, 15k, 16k.

Határozza meg *változó partíció* esetén a következő algoritmusok felhasználásával: *first fit*, *next fit*, *best fit*, *worst fit* a foglalási igényeknek megfelelő helyfoglalást.

Magyarázza az eredményeket.

Foglalási stratégiák - mintapélda 1

„1. Adott egy számítógépes rendszer (foglalási stratégiák),
melyben a következő

Szabad memória területek: 30k, 35k, 15k, 25k, 75k, 45k és

Foglalási igények: 39k, 40k, 33k, 20k, 21k áll rendelkezésre.

		Szabad területek					
Igény	Foglalható						

Foglalási stratégiák - mintapéldák 1

A rendszerben a memória *4 kbyte-os blokkokban* kerül nyilvántartásra, ennél kisebb méretű töredék igény esetén a teljes blokk lefoglalásra kerül.

a.) Határozza meg a *valódi foglalási igényt*!

b.) Határozza meg a foglalt/szabad memória teületeket *változó méretű partíció* esetén a következő algoritmusok felhasználásával: *first fit, next fit, best fit, worst fit* a foglalási igényeknek megfelelő helyfoglalást – táblázatos formában!

Foglalási stratégiák - mintapéldák 1

c.) Hasonlítsa össze és számolja ki, hogy a teljes szabad memóriaterület hány százaléka vész el átlagosan az egyes algoritmusok esetén!

A kapott eredményeket ábrázolja oszlop diagrammal!”

Meg kell határozni az *elvesző memóriát*. Hogyan?

Az elvesző memóriák az olyan blokkok, amelyek kisebbek, mint a legkisebb eddigi foglalásunk, ill. a fennmaradó blokkok azon részeit, ami a négyfel való osztási maradékaikat adja.”[1][2][3][4][5][6].

Irodalom

1. Vadász Dénes: Operációs rendszerek, ME, 2007.

<http://www.iit.uni-miskolc.hu/~vadasz/GEIAL302B/2007-Ea>

2. Operációs rendszerek

<https://dcs.uni-pannon.hu/oktatas/tantargyak/operacios-rendszerek>

3. Tannenbaum: Modern Operating Systems, Prentice Hall, 1992.

4. Kóczy, Kondorossi szerk: Operációs rendszerek mérnöki megközelítésben, Panem, 2000.

Irodalom

5. Operációs rendszerek

<https://vik.wiki.hu>

6. Heizlerné Bakonyi Viktória, Horváth László, Illés Zoltán, Nagy Tibor, Lutár Patrícia: Számítógépes alapismeretek II. ELTE 2012.

https://regi.tankonyvtar.hu/hu/tartalom/tamop412A/2010-0011_szamalap2/adatok.html

Memória kezelés - Szegmensek

Memória kezelés - Szegmensek

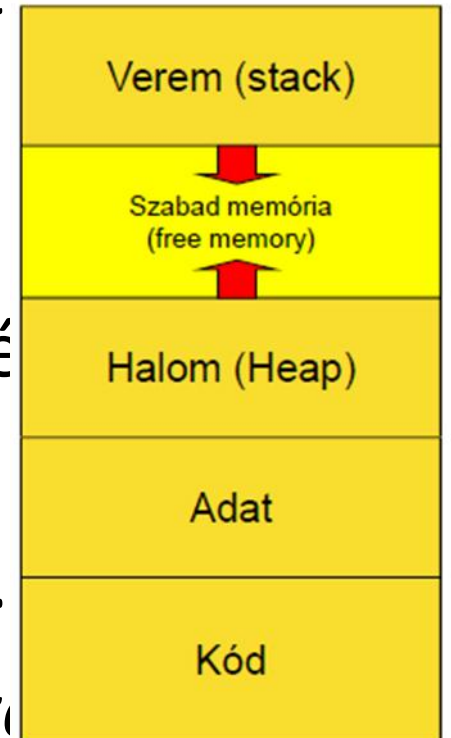
Memória kezelés - Szegmensek

A programozó:

- Adat, kód, bizonyos programkönyvtárak, stack, heap stb
- Nem egy összefüggő *lineáris memória területben* gondolkozik...

A szegmensszervezésű memória ennek az elképzelésnek a megvalósulása.

- A *logikai címtartomány* szegmensekre van osztva.
- A programozó egy *szegmenst* és azon belül egy *szegmens címet* ad meg.



Folyamat a memóriában

Memória kezelés - Szegmensek

A virtuális címtartomány nem egydimenziós.

Külön címtartományok

a kódnak, az adat szekcióknak, a veremnek, az osztott kódnak, kernel régióknak, stb.

Ezek a szegmensek.

Ez fordító támogatást igényel.

Memória kezelés – Szegmensek - allokálás

A virtuális memória *szegmensekre* van felosztva.

A szegmenseket *egyben lehet elhelyezni a memóriában vagy a háttértáron* kialakított részen.

Tehát, nem egyforma méretűek a blokkok.

Elhelyezés problémái: hasonlóak a *változó partíciós memória* kezelésnél tanultaknál, de a *szegmensek mérete a memória összes méretéhez képest nem olyan nagy* (mint ott egy partíció).

Memória kezelés - Szegmensek - allokálás

Elhelyezés:

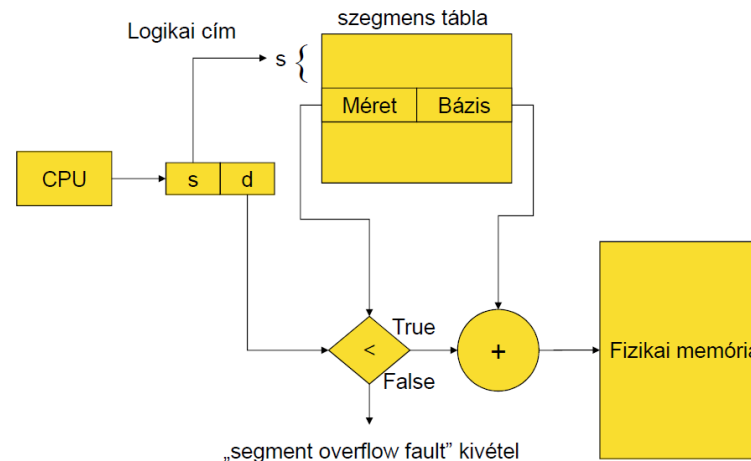
- Szegmens elhelyezése lyukakba.
- Elhelyezési algoritmusok: first fit / next fit / best fit / worst fit
- *Nyilvántartás szükséges a szabad területekről.*

Memória kezelés – Szegmensek - címképzés

Címképzés:

Virtuális cím: (s, d)

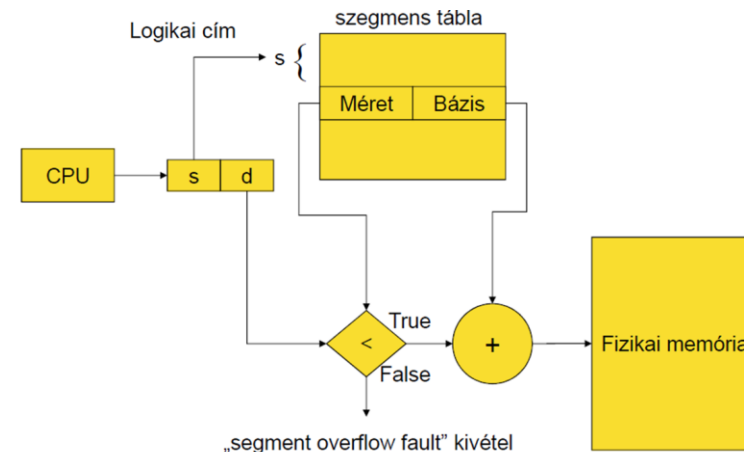
*Nyilvántartás kell a processzhez tartozó szegmensek elhelyezkedéséről - ez a **szegmenstábla**.*



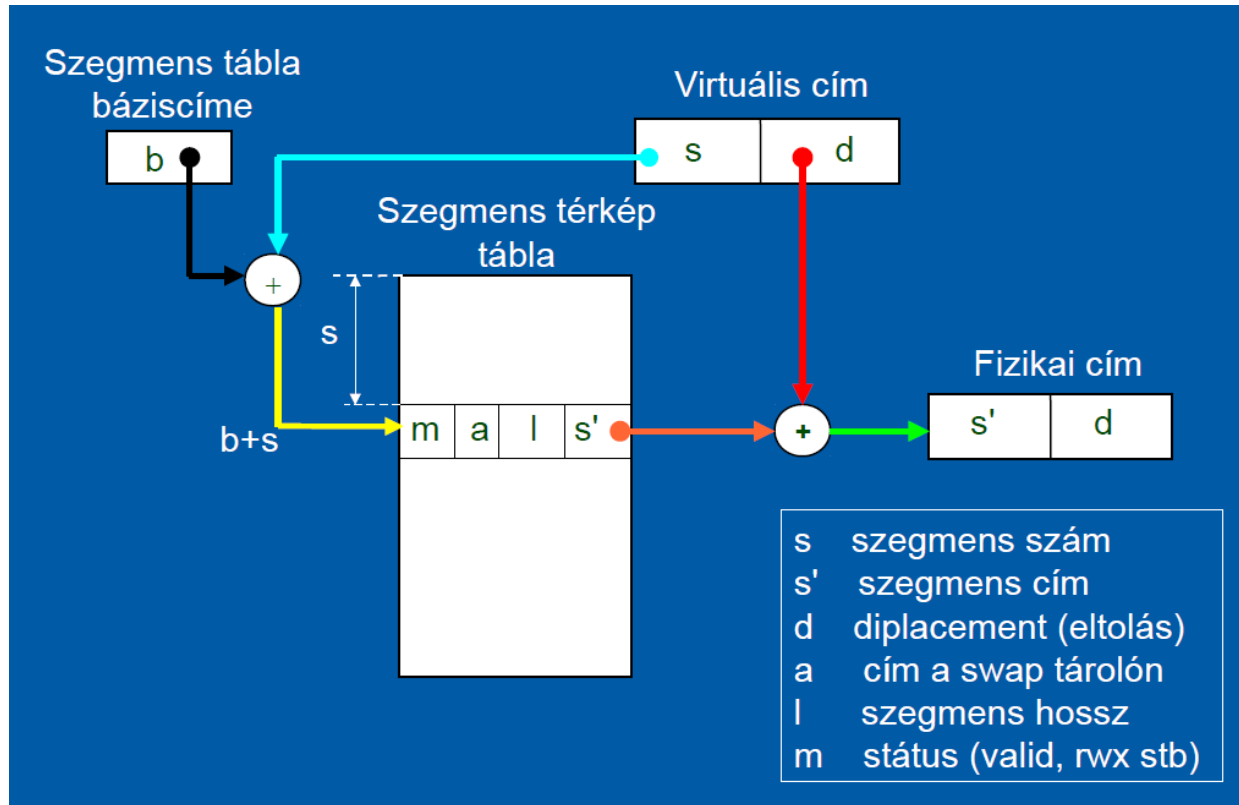
Memória kezelés - Szegmensek - címképzés

Ebben *egy bejegyzés*:

- szegmens *memóriában-e* vagy *sem*.
- szegmens elhelyezkedése a memóriában.
- szegmens *elhelyezkedése a háttértáron*.
- *szegmens hossz*.
- egyéb adatok (pl. r, rw).



Szegmensenkénti címleképzés



Forrás: VadászD

Memória kezelés – Szegmensek - Allokálás

Szegmenshiba: ugyanaz, mint a laphiba.

Szegmenshiba kezelése ugyanúgy, mint lapozásos memória kezelésnél.

Védelem

Szegmensen *belüli eltolás* és *szegmenshossz összevetése*.

Belső védelem (védelem saját magától, hibáktól): szegmensekhez jogosultságok hozzárendelésével (pl. kódszegmensek csak olvashatóak)

Memória kezelés - szegmentálásos virtuális memória

SHM

Osztott szegmensek használatával, a lapozásoshoz hasonlóan.

Előnyösebb, mint osztott lapok, mert logikailag összetartozó adatok vannak egy szegmensben.

Memória kezelés - szegmentálásos virtuális memória

Előnyök

- Kevesebb *a szegmenshiba*, hiszen a szegmensekre osztás igazodik a programok logikai szerkezetéhez (lokalitás elv)!
- Kényelmesebb osztott memória használat.
- **Hátrányok**
 - Sokkal több munkája van az operációs rendszernek, mint a lapozásosnál: *elhelyezési stratégia, bonyolultabb nyilvántartás*
 - Van külső elaprózódás. A helykihasználás kisebb, mint a lapozásosnál.

Memória kezelés - Szegmentálásos és lapozásos virtuális memória

Szegmentálásos és lapozásos virtuális memória

A két módszer ötvözése:

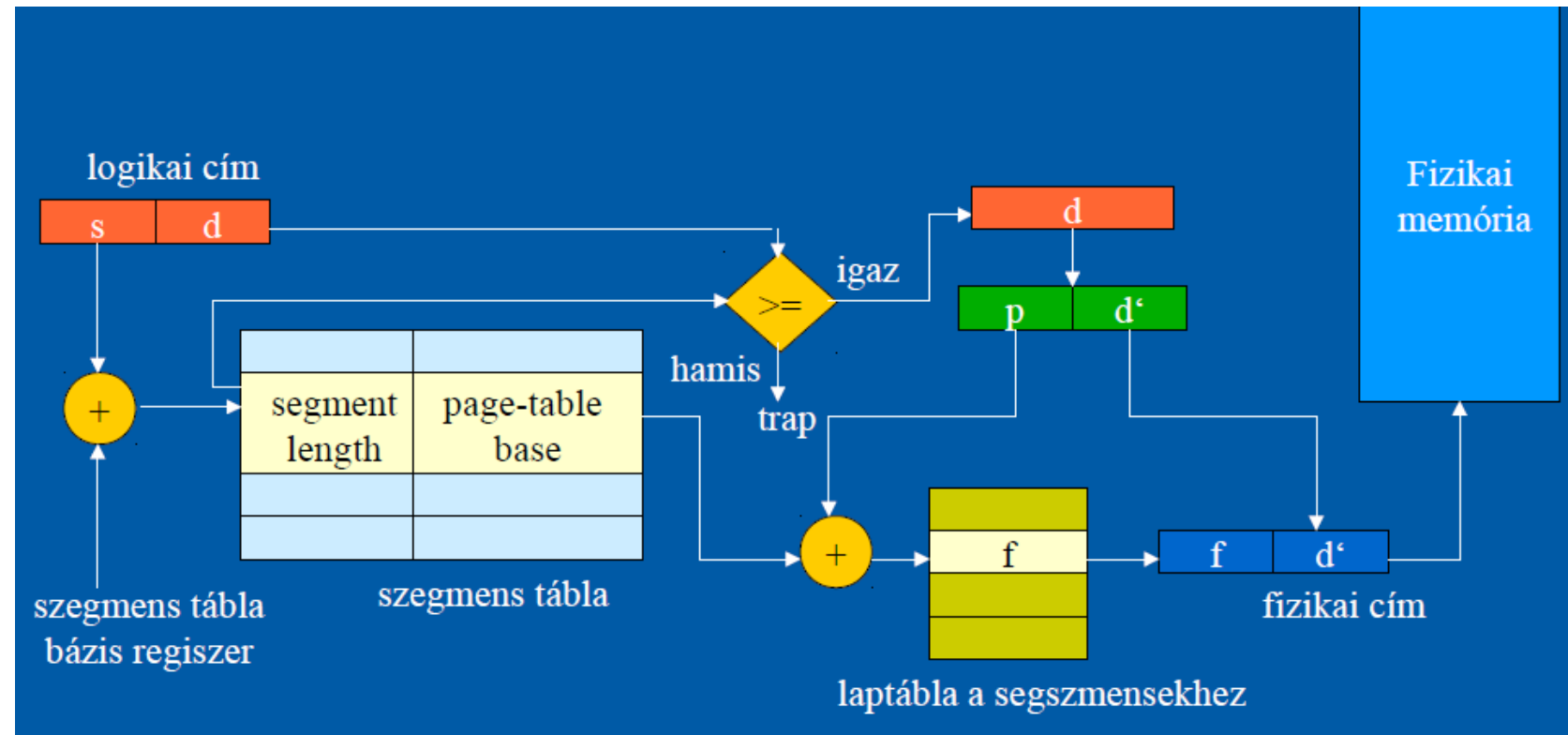
- A processz memória területe szegmensekre van osztva.
- A szegmenseket kell elhelyezni vagy a memóriában vagy a háttértáron.
- Az elhelyezés nyilvántartása a processzhez tartozó *szegmenstáblában*. (szegmentálás)

Memória kezelés- Szegmentálásos és lapozásos virtuális memória

- *Előnyei:* kevesebb szegmenshiba, osztott szegmensek
- A szegmensek memóriában elhelyezésekor lapokra bontás.
(lapozás előnyei: nincs elhelyezési stratégia, bonyolult lyuk nyilvántartás, külső elaprózódás)

Memória kezelés - Szegmentálós és lapozós virtuális memória

Minden memóriában levő szegmenshez tartozik egy laptábla.
Bonyolultabb címképzés:



Forrás: VadászD

WinNT memória kezelése

Virtuális, lapozós

- *Két szintes laptáblát + VAD-ot (V.A.S) használ a lapok nyilvántartásához (Virtual Address Descriptor).*
- *Lapkeret adatbázist használ a szabad lapkeretek kereséséhez.*
- *Munkakészletet használ, igény szerinti belapozós.*
- *Önálló taszk a kilapozó.*

WinNT memória kezelése

- 32 bites címzés: 2^{32} lineáris címtartomány (4GB)
- A felső 2GB rendszercímtartomány, kernel módban látható (Pentium 1GB).
- Az alsó 2GB taszkonként egyedi címtartomány (Pentium 3GB).

Linux memória kezelése

- Szegmentáció nincs (de ettől még a szegmens regisztereknek a CPU-ban vannak értékeik!)
- *Lapozós virtuális memória*
 - 4 KByte méret (4 MByte is támogatott)
 - 64-bit címtartomány
 - 3 szintű laptábla

Linux memória kezelése

- x86-on:
 - Igazából itt csak 2 szintet használ
 - (64-biten pedig 4 szintet (48-bit))
 - Lapozás HW támogatott, TLB is
- Igény szerinti belapozás

Linux memória kezelése

- Laptáblák
 - Global Directory
 - Middle Directory
 - Page Table
 - Offset

User mód/Kernel mód memória rész.

Linux memória kezelése

- A lapozás miatt nem gond a darabos memória, de mégis hatékonyabb az egybefüggő memóriarész (DMA műveletek, TLB hatékonyság)
- Törekszik a folyamatos lapkeretek allokációjára.

Linux memória kezelése

Processz szemszögéből:

- Processz logikai címekkel dolgozik.
- Címtérület – az egész ami használható.
- Ez minden processznek külön.
- Memória régiók – az allokálás „darabjai”
 - Mérete a lapméret többszöröse
 - fork, exec, malloc (brk), mmap, shmat

Linux memória kezelése

- Memória deszkriptor (processz kontextusban)
 - Mindenféle adatok...
 - Konkrét memória részek egy láncolt listán
 - Darabka leírója + next pointer

Linux memória kezelése

- Processz szemszögéből
 - Foglalásnál nem feltétlen rendelődik egyből fizikai memória is processzhez.
- Laphiba esetén:
 - Elérni kívánt cím összehasonlítása a címterülettel
 - Ha nincs benne → hibás elérés (SEGV)
 - Tényleges fizikai memória foglалás
 - Swap!

Linux memória kezelése

- CoW – Copy on Write
 - Nem készül tényleges másolat, csak hivatkozás.
 - De, amint írás történik, készül egy másolat, és oda történik az írás.
 - pl. fork()-nal”[1][2][3][4][5][6]

Irodalom

1. Vadász Dénes: Operációs rendszerek, ME, 2007.

<http://www.iit.uni-miskolc.hu/~vadasz/GEIAL302B/2007-Ea>

2. Operációs rendszerek

<https://dcs.uni-pannon.hu/oktatas/tantargyak/operacios-rendszerek>

3. Tannenbaum: Modern Operating Systems, Prentice Hall, 1992.

4. Kóczy, Kondorossi szerk: Operációs rendszerek mérnöki megközelítésben, Panem, 2000.

Irodalom

5. Operációs rendszerek

<https://vik.wiki.hu>

6. Heizlerné Bakonyi Viktória, Horváth László, Illés Zoltán, Nagy Tibor, Lutár Patrícia: Számítógépes alapismeretek II. ELTE 2012.

https://regi.tankonyvtar.hu/hu/tartalom/tamop412A/2010-0011_szamalap2/adatok.html